

Analytics Engineer Challenge

Table des matières

Table des matières	2
Résumé exécutif	4
Alignement avec les exigences du challenge	4
Livrables et références associées	4
Conception recommandée du Dashboard Executive Growth Allocation	4
Positionnement du dictionnaire de données	5
PARTIE I — Documentation des enrichissements de données 1	6
1.1 Dataset initial — état des lieux	6
1.2 La question centrale comme boussole	6
1.3 Principe directeur Senior Analytics Engineer	6
2. Données structurées — Nouvelles tables	7
2.1 FlightCosts — Coûts opérationnels par vol	7
Pourquoi cette table est indispensable	7
Documentation colonne par colonne	7
Hypothèses de calcul documentées	8
2.2 LoyaltyActivity — Transactions fidélité	8
Pourquoi cette table est indispensable	8
Documentation colonne par colonne	8
2.3 AncillaryDetail — Détail des revenus annexes	9
Pourquoi cette table est indispensable	9
Documentation colonne par colonne	9
3. Données non structurées	10
3.1 CustomerReviews — Avis clients texte libre	10
Pourquoi cette source est indispensable	10
Documentation colonne par colonne	10
3.2 SupportTickets — Tickets de support client	11
Pourquoi cette source est indispensable	12
Documentation colonne par colonne	12
3.3 ComplaintLogs — Journal de plaintes formelles	13
Pourquoi cette source est indispensable	13
Documentation colonne par colonne	13
4. Enrichissements dans les tables existantes	14
4.1 Table Bookings — Colonnes ajoutées	14
4.2 Table Flights — Colonne ajoutée	15
5. Synthèse et couverture analytique	15
5.1 Couverture des hypothèses par dataset	15
5.2 Questions décisionnelles débloquées	15

PARTIE II — Justification d'architecture et décisions techniques	17
1. Contexte et contraintes du projet	17
2. Choix de stack technologique	17
3. Choix d'architecture : Hybrid Model	17
4. Justification du Raw Vault.....	18
4.1 Pourquoi des Hubs ?.....	18
4.2 Pourquoi des Links ?	18
4.3 Pourquoi des Satellites avec hash_diff ?	18
5. Choix SCD2 : dbt Snapshots vs Incremental.....	18
6. Justification du Business Vault	18
6.1 Point-In-Time (PIT)	18
6.2 Bridge Tables.....	18
7. Justification du Star Schema Mart.....	19
8. Justification de la Semantic Layer.....	19
9. Justification des OBT AI-Ready	19
10. Justification de la couche Embeddings (Partie 4)	19
11. Trade-offs et limites documentées	20
12. Ordre d'exécution dbt	20
PARTIE III — Architecture data, Power BI et Agentic AI.....	21
1. Objectif du document.....	21
2. Contexte metier	21
3. Vue d'ensemble de l'architecture	21
4. Justification des couches data	21
Synthese des couches.....	21
5. Pourquoi utiliser des OBT	22
6. Choix du modele Power BI.....	22
Tables Power BI recommandees	22
7. Role de la couche Agentic AI	22
8. Fonctionnement de l'Agentic AI.....	22
Composants Agentic AI	23
9. Outils analytiques disponibles.....	23
10. Flux technique de bout en bout	23
11. Recherche semantique et embeddings.....	23
12. Qualite, gouvernance et tracabilite.....	23
13. Limites et precautions	23
14. Evolution possible	24
15. Conclusion	24

Résumé exécutif

Ce dossier présente une solution analytique complète pour aider Air Côte d’Ivoire à arbitrer son budget de croissance entre trois leviers : optimisation du réseau, rétention client et développement des revenus additionnels. La logique du document est volontairement décisionnelle : chaque donnée ajoutée, chaque couche de modélisation et chaque composant IA est justifié par son impact sur une recommandation d’investissement.

Le projet combine des données structurées, des signaux non structurés, un modèle analytique robuste, une couche sémantique business-friendly, des règles décisionnelles de type ontology et une interface Agentic AI capable d’interroger les données de manière outillée et contrôlée.

Alignement avec les exigences du challenge

Exigence du brief	Réponse dans le projet	Statut
Business understanding	Domaines réseau, opérations, commercial, client et ancillaires explicités.	Couvert
Data generation / enrichissement	FlightCosts, LoyaltyActivity, AncillaryDetail, CustomerReviews, SupportTickets, ComplaintLogs.	Couvert
Structured + unstructured data	Intégration des coûts, fidélité, ancillaires, avis, tickets et plaintes.	Couvert
Data modeling	Architecture hybride Data Vault + Business Vault + Star Schema Mart.	Couvert
Semantic layer	Vues sémantiques pour les KPIs opérationnels, financiers, clients et réseau.	Couvert
Ontology / reasoning rules	Labels décisionnels : routes stratégiques, clients à risque, opportunités upsell.	Couvert
Dashboard	Modèle Power BI orienté Executive Growth Allocation.	À documenter avec captures
Agentic AI	Interface React/Python, outils analytiques, DuckDB, Cohere et recherche sémantique.	Couvert
Data dictionary	Documenté dans le fichier Excel DATA_DICTIONARY.xlsx.	Fourni séparément
Demo / screenshots	À joindre au dossier final : captures Power BI et vidéo IA.	À compléter

Livrables et références associées

Les éléments suivants constituent le dossier complet de soumission :

- Code source du pipeline dbt, DuckDB et scripts Python.
- README d’installation, de configuration et d’exécution.
- Dictionnaire de données Excel : DATA_DICTIONARY.xlsx.
- Diagramme de modélisation : sources, staging, vault, marts, semantic, ontology et AI-ready.
- Dashboard Power BI avec pages réseau, rétention, upsell et recommandations.
- Captures ou courte vidéo de walkthrough du dashboard.
- Interface Agentic AI et vidéo de démonstration des questions métier.
- Présent document : hypothèses, architecture, limites, prochaines étapes et justification des choix.

Conception recommandée du Dashboard Executive Growth Allocation

Page	Objectif décisionnel
Page 1 - Executive Summary	Budget allocation, priorités d’investissement, synthèse des gains attendus.
Page 2 - Network & Profitability	Revenus, marge, yield, load factor, OTP, cancellations, route opportunity matrix.
Page 3 - Customer Retention	Segments, clients high-value à risque, loyalty engagement, satisfaction

	et complaint themes.
Page 4 - Upsell / Cross-sell	Ancillary attach rate, upgrade conversion, propension par segment, revenue per pax.
Page 5 - Unstructured Signals	Sentiment, plaintes, tickets, thèmes faibles par route et causes racines.
Page 6 - Final Recommendations	Routes à développer, routes à défendre, clients à retenir, offres à prioriser.

Positionnement du dictionnaire de données

Le dictionnaire de données complet est fourni séparément dans le fichier Excel DATA_DICTIONARY.xlsx. Il documente les tables, colonnes, types, clés, descriptions métier, règles de calcul, couches dbt associées et usages analytiques. Le présent document le référence sans le dupliquer afin de conserver un dossier lisible et orienté présentation jury.

PARTIE I : Documentation des enrichissements de données

Domaines existants et stratégiques

Réseau (Network) : C'est le cœur stratégique. Il définit où et quand la compagnie vole (choix des routes, des fréquences, hubs). La BI y analyse la rentabilité des lignes et la connectivité.

Opérations (Operations) : La gestion des vols au quotidien (ponctualité, utilisation de la flotte, gestion des équipages, maintenance). C'est le domaine le plus critique pour les coûts opérationnels et la réputation.

Commercial : L'analyse de l'acquisition, de la performance des canaux de distribution (Web vs. Agences) et de la segmentation des passagers.

Client (Customer Experience) : Le suivi du parcours passager (fidélité, satisfaction, réclamations). Essentiel pour maximiser la Customer Lifetime Value (LTV).

Revenus Annexes (Ancillary Revenues) : Tout ce qui est vendu en plus du billet (bagages, sièges, repas, assurances). C'est souvent là que se joue la rentabilité nette d'une compagnie moderne.

1.1 Dataset initial — état des lieux

Le dataset de départ d'Air Côte d'Ivoire est constitué de 5 tables :

Table	Lignes	Contenu	Lacune principale
Airports	~15	Codes, coordonnées, timezones	Aucune — table de référence complète
Routes	~10	Réseau, distances, block_time	Aucun coût, aucune donnée compétitive
Customers	300	Profils, segments, tiers fidélité	Pas d'historique fidélité, pas de CLV
Flights	480	Vols planifiés et réels, OTP	Aucun coût opérationnel
Bookings	~1500	Réservations, tarifs, ancillaires globaux	Pas de détail par type d'ancillaire

1.2 La question centrale comme boussole

"Where should Air CI invest first to maximize profitable growth: route expansion & optimization, customer retention, or upsell/cross-sell?"

Cette question implique de **comparer trois hypothèses d'investissement concurrentes**. Chaque enrichissement est justifié uniquement s'il permet de mesurer, affiner ou trancher entre ces trois options :

Hypothèse	Description	Condition de supériorité
H1 — Réseau	Route expansion & optimization	Des routes rentables existent ou des routes actuelles sont sous-optimisées
H2 — Rétention	Customer retention	Le churn est élevé — retenir vaut plus qu'acquérir ou étendre
H3 — Upsell	Upsell / Cross-sell	La base client est stable mais sous-monétisée (ancillaire/rev < benchmark)

1.3 Principe

Règle d'or : Un dataset ou une colonne n'est pas ajouté(e) parce qu'il/elle 'enrichit l'analyse en général'. Il/elle est ajouté(e) parce qu'il/elle débloque une question précise qui oriente le choix entre H1, H2 et H3. Sans cette règle, on produit du reporting, pas de l'analytics décisionnel.

Deux types d'enrichissements ont été retenus :

- Nouvelles tables séparées pour les données coût, fidélité et détail ancillaire — données modèles recalculables ou historiques de transactions
- Nouvelles colonnes dans tables existantes pour les attributs bruts de réservation (flags d'achat) et de vol (passagers embarqués)

Important : Les métriques calculées (CLV, churn_risk_score) ne sont PAS ajoutées dans les tables sources. Elles vivent dans la couche analytique (marts dbt) pour éviter la confusion entre données brutes et outputs analytiques.

2. Données structurées — Nouvelles tables

2.1 FlightCosts — Coûts opérationnels par vol

Hypothèse débloquée : H1 — Route expansion & optimization

Pourquoi cette table est indispensable

Le dataset initial contient les **revenus** (ticket_price_usd + ancillary_revenue_usd) mais aucune donnée de **coût**. Or la question centrale parle de **profitable growth**. Sans coût, il est impossible de calculer une marge. Une route avec 80% de load factor peut être déficitaire si le mauvais appareil est assigné.

Exemple concret : un A330-900 coûte ~12 000\$/h contre ~2 800\$/h pour un ATR72. Sur une route de 350km à 50 minutes de vol (ABJ-BYK), assigner un A330 au lieu d'un ATR72 multiplie le coût opérationnel par 4,3. Sans cette table, on ne peut pas détecter cette erreur d'allocation.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
flight_id	Clé étrangère (TEXT)	Lien avec la table Flights. Permet la jointure pour calculer marge = revenue - cost par vol.	Marge par vol, Marge par route	H1 — Réseau
route_id	Clé étrangère (TEXT)	Dénormalisation pour faciliter les agrégations par route sans double jointure.	Marge nette par route	H1 — Réseau
aircraft_type	Dimension (TEXT)	Permet d'analyser l'adéquation flotte-réseau. Identifie les routes où le mauvais avion est assigné.	Coût/siège par type d'avion	H1 — Réseau
fuel_cost_usd	Mesure (DECIMAL)	Composante la plus variable du coût (40-50% du total). Calculé : distance_km × 3.1L/km × 0.84\$/L ± 10%.	Coût opérationnel total, Marge par route	H1 — Réseau
crew_cost_usd	Mesure (DECIMAL)	35% du coût horaire avion × block_time_h. Varie selon la durée du vol et le type d'appareil.	Coût opérationnel total	H1 — Réseau
airport_handling_usd	Mesure (DECIMAL)	Frais aéroport fixes (420\$ base) + variable par passager. Permet de comparer les coûts sol entre aéroports.	Coût sol par rotation	H1 — Réseau
maintenance_usd	Mesure (DECIMAL)	15% du coût horaire avion × block_time. Coût réel de la flotte souvent sous-estimé dans les analyses revenue-only.	Coût total vol, rentabilité flotte	H1 — Réseau
total_cost_usd	Mesure calculée (DECIMAL)	Somme des 4 composantes. Colonne dénormalisée pour simplifier les requêtes analytiques de marge.	Marge nette = revenue - total_cost	H1 — Réseau
cost_per_seat_usd	Mesure calculée (DECIMAL)	total_cost / seat_capacity. Permet de comparer l'efficacité opérationnelle entre types d'avion et entre routes	CASK (Cost per Available Seat Km)	H1 — Réseau

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
		indépendamment de la taille de l'appareil.		

Hypothèses de calcul documentées

Composante	Formule retenue	Justification	Tolérance
fuel_cost_usd	distance_km × 3.1 L/km × 0.84 \$/L	Conso moyenne long-courrier Airbus. Prix carburant Jet A1 Afrique 2025.	±10%
crew_cost_usd	coût_horaire_avion × 35% × block_time_h	Convention IATA : coût crew ≈ 30-40% du coût opérationnel direct.	±8%
airport_handling_usd	420\$ fixe + variable_pax	Benchmark handling Afrique de l'Ouest (ABJ, ACC, LOS).	±15%
maintenance_usd	coût_horaire_avion × 15% × block_time_h	Ratio maintenance / coût opérationnel direct pour flotte Airbus/ATR.	±12%

2.2 LoyaltyActivity — Transactions fidélité

Hypothèse débloquée : H2 — Customer retention

Pourquoi cette table est indispensable

La table Customers contient le champ **loyalty_tier** (Gold, Silver, Explorer) mais c'est un **état statique**. On sait qu'un client est Gold aujourd'hui, mais pas comment il l'est devenu ni si son engagement diminue. Sans l'historique des transactions, il est impossible de calculer le churn rate réel par tier ou de détecter un client Gold en train de se désengager.

Exemple concret : un client Gold sans transaction fidélité depuis 4 mois a 3× plus de probabilité de cherner qu'un Gold actif. Cette information n'est visible que dans l'historique de transactions, pas dans un snapshot statique.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
loyalty_id	Clé primaire (TEXT)	Identifiant unique de chaque événement de transaction fidélité.	N/A — identifiant	H2 — Rétention
customer_id	Clé étrangère (TEXT)	Lien avec Customers. Permet de croiser comportement fidélité avec segment et tier pour le scoring churn.	Churn rate par tier, CLV	H2 — Rétention
event_date	Date (DATE)	Permet d'analyser la vélocité d'accumulation, de détecter les périodes d'inactivité et de calculer la récence (composante RFM).	Loyalty engagement score, Recency	H2 — Rétention
event_type	Dimension (TEXT)	Distingue les gains (vol, partenaire), les rédemptions (upgrade, lounge) et les mouvements de tier. Révèle si le programme est utilisé activement.	Taux de rédemption, Tier progression rate	H2 + H3

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
points_earned	Mesure (INTEGER)	Volume de points générés par transaction. Corrèle avec la valeur du billet acheté (points = ticket_price × 10).	Loyalty revenue contribution	H2 — Rétention
points_redeemed	Mesure (INTEGER)	Points dépensés. Un membre qui rachète est engagé — un membre qui accumule sans jamais racheter est à risque de churn ou d'expiration.	Redemption rate, Programme ROI	H2 — Rétention
balance_after	Mesure (INTEGER)	Solde après transaction. Permet de segmenter les membres par niveau d'accumulation et de détecter les balances à risque d'expiration.	Points liability, Churn early warning	H2 — Rétention
loyalty_tier	Dimension (TEXT)	Tier au moment de la transaction. Permet de suivre les changements de tier dans le temps — montée (upsell de valeur) ou descente (signal churn).	Tier stagnation rate, Tier progression	H2 — Rétention

2.3 AncillaryDetail — Détail des revenus annexes

Hypothèse débloquée : H3 — Upsell / Cross-sell

Pourquoi cette table est indispensable

La table Bookings contient **ancillary_revenue_usd** comme montant global. On sait qu'un client a payé 48\$ d'ancillaires mais pas si c'est un bagage supplémentaire, un siège prioritaire, un repas ou une assurance. Sans ce détail, il est impossible de savoir **quel produit ancillaire pousser à quel segment**. Pousser un bagage supplémentaire à un client qui voyage toujours en bagage cabine est une opportunité manquée.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
ancillary_id	Clé primaire (TEXT)	Identifiant unique de chaque achat ancillaire. Permet une granularité ligne par ligne.	N/A — identifiant	H3 — Upsell
booking_id	Clé étrangère (TEXT)	Lien avec Bookings. Une réservation peut avoir plusieurs ancillaires (bagage + siège + repas). Relation one-to-many.	Ancillary attach rate, Rev/pax	H3 — Upsell
customer_id	Clé étrangère (TEXT)	Lien direct avec Customers pour croiser le comportement d'achat ancillaire avec le segment et le tier.	Propensity score par segment	H3 — Upsell
ancillary_type	Dimension (TEXT)	Type de service : Baggage / Seat Selection / Meal / Insurance / Lounge / Priority Boarding. C'est la colonne la plus importante — permet de savoir QUOI vendre à	Attach rate par type et segment	H3 — Upsell

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
		QUI.		
ancillary_amount_usd	Mesure (DECIMAL)	Revenu généré par achat ancillaire. Permet de calculer le mix revenue par type et d'identifier les produits les plus rentables.	Ancillary revenue per pax, Revenue mix	H3 — Upsell
purchased_at	Dimension (TEXT)	Moment d'achat : At Booking / Online Check-in / Airport / On-board. Révèle le meilleur moment pour proposer chaque ancillaire selon le segment.	Conversion rate par canal temporel	H3 — Upsell
route_type	Dimension (TEXT)	Domestic / Regional / International. L'appétit pour les ancillaires varie fortement selon la longueur du vol (ex. repas inutile sur 50 min, indispensable sur 8h).	Attach rate par type de route	H3 — Upsell

3. Données non structurées

Trois sources non structurées ont été retenues. Le critère de sélection est identique : chaque source doit produire un signal analytique qui oriente le choix entre H1, H2 et H3. Les notes opérationnelles internes ont été **explicitement exclues** car elles adressent la performance opérationnelle interne, pas la question de croissance rentable côté client.

Source	Volume	Taux base	Hypothèse principale
CustomerReviews	80 avis	27% des 300 customers	H1 + H2 + H3
SupportTickets	120 tickets	40% des 300 customers	H2 + H3
ComplaintLogs	150 plaintes	10% des bookings (~1500)	H1 + H2 + H3

3.1 CustomerReviews — Avis clients texte libre

Hypothèses débloquées : H1 (réputation route) · H2 (pourquoi le client part) · H3 (friction ancillaire)

Pourquoi cette source est indispensable

C'est la seule source qui explique **POURQUOI** un KPI se dégrade, pas seulement QU'IL se dégrade. L'OTP (On-Time Performance) dans la table Flights indique un retard, mais pas son impact sur la demande future. Un avis client négatif sur une route spécifique est un signal avancé d'érosion de la demande — il précède la baisse de réservations de plusieurs semaines.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
review_id	Clé primaire (TEXT)	Identifiant unique de chaque avis. Permet le suivi et la déduplication.	N/A — identifiant	—
customer_id	Clé étrangère	Lie l'avis au profil client	Sentiment score	H2 — Rétention

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
	(TEXT)	(segment, tier). Permet de pondérer un avis négatif d'un Gold 5× plus qu'un Standard.	pondéré par valeur client	
flight_id	Clé étrangère (TEXT)	Lie l'avis à un vol spécifique. Permet de corréler le sentiment avec l'OTP, le type d'avion, le statut du vol.	Corrélation OTP / Sentiment	H1 — Réseau
route_id	Clé étrangère (TEXT)	Agrégation des avis par route. Permet de calculer un score de réputation par route — signal clé pour H1.	Sentiment score par route	H1 — Réseau
review_date	Date (DATE)	Permet d'analyser l'évolution de la satisfaction dans le temps. Détecte les dégradations saisonnières ou post-incident.	Tendance satisfaction MoM	H2 — Rétention
rating_1_5	Mesure (INTEGER)	Score quantitatif 1-5. Permet des agrégations rapides (AVG rating par route, par segment). Complément du NLP.	Rating moyen par route et segment	H1 + H2
review_text	DONNÉE NON STRUCTURÉE (TEXT)	Texte libre bilingue FR/EN. Source principale du NLP. Révèle les causes réelles d'insatisfaction (retard, service, confort, prix, bagages).	NLP sentiment score, Topic extraction	H1 + H2 + H3
nlp_sentiment	Score calculé (TEXT)	Pré-calculé : Positif / Neutre / Négatif. Accélère les analyses sans pipeline NLP en temps réel. À recalculer avec VADER ou mBERT en production.	Sentiment score par route/segment	H1 + H2 + H3
complaint_category	Label NLP (TEXT)	Catégorie extraite : Retard, Bagages, Service, Confort, Prix. Permet de cibler les actions correctives par type de problème.	Distribution des problèmes par route	H1 + H2 + H3
language	Dimension (TEXT)	FR / EN. Important pour choisir le bon modèle NLP et pour la segmentation géographique (FR = Afrique de l'Ouest, EN = diaspora/business).	Segmentation géolinguistique	H2 — Rétention
linked_booking_id	Clé étrangère (TEXT)	Lie l'avis à la réservation. Permet de croiser satisfaction avec le canal d'achat, la classe tarifaire et les ancillaires achetés.	Satisfaction par fare_family, par canal	H2 + H3

3.2 SupportTickets — Tickets de support client

Hypothèses débloquées : H2 (churn post-friction) · H3 (produit ancillaire défaillant)

Pourquoi cette source est indispensable

Les tickets de support capturent la **friction en temps réel**. Contrairement aux avis (post-vol, rétrospectifs), un ticket est ouvert pendant ou juste après un problème, avec une intention d'action de la part du client. Un client Gold qui ouvre un ticket Urgent non résolu a une probabilité de churn **3× supérieure** à la moyenne. Cette source est complémentaire aux ComplaintLogs : le ticket est la demande client, le complaint log est la réponse formelle de la compagnie.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
ticket_id	Clé primaire (TEXT)	Identifiant unique. Permet le suivi et la mesure du volume de contacts.	Volume tickets par période	—
customer_id	Clé étrangère (TEXT)	Lien avec Customers. Permet de pondérer l'urgence par tier (Gold = priorité haute) et de calculer le churn risk post-ticket.	Churn rate par tier post-ticket	H2 — Réten
booking_id	Clé étrangère (TEXT)	Lie le ticket à une réservation spécifique. Permet de corréler friction avec le canal d'achat, la classe et les ancillaires.	Friction par canal / fare_family	H2 + H3
created_date	Date (DATE)	Permet de mesurer la saisonnalité des incidents et de calculer le délai entre vol et contact (indicateur de gravité perçue).	Délai vol-contact, Volume MoM	H2 — Réten
priority	Dimension (TEXT)	Low / Medium / High / Urgent. Corrèle avec le tier client et le type de problème. Permet de prioriser le backlog support.	SLA compliance, Backlog urgent	H2 — Réten
category	Dimension (TEXT)	Type de problème : Bagage, Retard, Remboursement, Service bord, Upgrade/Ancillaire, Info vol, Fidélité. La catégorie 'Upgrade/Ancillaire' est le signal direct pour H3 — un produit ancillaire défaillant freine l'upsell futur.	Distribution problèmes, Attach rate post-résolution	H2 + H3
ticket_text	DONNÉE NON STRUCTURÉE (TEXT)	Texte libre de la demande client. NLP extrait : émotion (frustration, urgence), produit concerné, intention de churn explicite ('je ne volerai plus avec vous').	Churn intent detection, Topic analysis	H2 + H3
status	Dimension (TEXT)	Résolu / En cours / Escaladé / Fermé sans suite. Le statut 'Escaladé' et 'Fermé sans suite' sont les signaux les plus forts de churn imminent.	Resolution rate, Churn risk score	H2 — Réten
resolution_days	Mesure (INTEGER)	Délai de résolution. Un délai > 7 jours sur un ticket High/Urgent est corrélé avec une probabilité de churn élevée et un avis négatif ultérieur.	SLA performance, Corrélation délai/churn	H2 — Réten
customer_tier	Dimension	Tier du client au moment du	Revenue at risk par	H2 — Réten

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
	(TEXT)	ticket. Permet de prioriser le traitement (Gold > Silver > Explorer) et de mesurer l'impact financier d'un churn par tier.	tier	

3.3 ComplaintLogs — Journal de plaintes formelles

Hypothèses débloquées : H1 (qualité opérationnelle route) · H2 (churn post-plainte) · H3 (friction ancillaire)

Pourquoi cette source est indispensable

Contrairement aux tickets (demande client) et aux avis (opinion post-vol), les complaint logs sont des **incidents documentés formellement** avec un département responsable et une action corrective. C'est la source qui permet de mesurer si la compagnie **agit réellement** sur les problèmes signalés. Le champ **corrective_action_taken** est une donnée non structurée unique : traité en NLP, il révèle les types d'actions les plus efficaces pour prévenir les récidives.

Documentation colonne par colonne

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
complaint_id	Clé primaire (TEXT)	Identifiant unique. Permet le tracking et la mesure du volume de plaintes formelles.	Volume plaintes par période et route	—
customer_id	Clé étrangère (TEXT)	Lien avec Customers. Permet de mesurer le taux de plainte par segment et de pondérer la gravité par tier.	Plainte rate par segment et tier	H2 — Rétention
flight_id	Clé étrangère (TEXT)	Lien avec Flights. Permet de corrélérer les plaintes avec l'OTP, le type d'avion et le statut du vol.	Plaintes par vol, corrélation OTP/plainte	H1 + H2
booking_id	Clé étrangère (TEXT)	Lien avec Bookings. Permet de corrélérer la plainte avec la classe tarifaire, le canal d'achat et les ancillaires.	Plainte rate par fare_family et canal	H2 + H3
complaint_date	Date (DATE)	Analyse temporelle : saisonnalité des plaintes, corrélation avec des événements opérationnels (ouverture de route, changement d'avion).	Trending plaintes MoM, détection anomalies	H1 + H2
complaint_type	Dimension (TEXT)	Delay / Baggage / Service / Ramp Ops / Others. Permet d'identifier les routes avec des problèmes opérationnels systémiques (ex. ABJ-CDG : 60% Delay = problème de rotation).	Distribution types par route, OTP root cause	H1 + H2
complaint_text	DONNÉE NON STRUCTURÉE	Texte libre de la plainte. NLP extrait : sévérité émotionnelle, produit ou service concerné,	Churn intent, Competitor mention, Severity	H1 + H2 + H3

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
	(TEXT)	mention explicite de churn ('je ne reviendrai pas'), comparaison concurrentielle ('Ethiopian est mieux').		
assigned_department	Dimension (TEXT)	Customer Service / Operations / Ramp / Baggage. Permet de mesurer la charge par département et d'identifier les services sous-dimensionnés.	Workload par département, SLA par dept	H2 — Rétention
resolution_status	Dimension (TEXT)	Resolved / Pending / Escalated / Closed No Action. 'Closed No Action' est le signal le plus dangereux : client plaignant non résolu = churn quasi-certain + avis négatif probable.	Resolution rate, Churn risk post-plainte	H2 — Rétention
corrective_action_taken	DONNÉE NON STRUCTURÉE (TEXT)	Texte libre décrivant l'action corrective. C'est la colonne la plus analytiquement riche de la table : NLP extrait le type d'action (compensation financière, formation équipe, fix process) et permet de mesurer quelle action réduit le mieux la récurrence.	Efficacité des actions correctives, ROI résolution	H2 + H1

4. Enrichissements dans les tables existantes

Ces colonnes sont des **attributs bruts observés** qui appartiennent naturellement à la table mère — contrairement aux métriques calculées (CLV, churn score) qui vivent dans la couche analytique. Elles suivent l'approche proposée initialement et conservée car analytiquement correcte.

4.1 Table Bookings — Colonnes ajoutées

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
insurance_purchase_flag	Booléen (0/1)	Indique si le client a acheté une assurance voyage. Signal cross-sell direct. Permet de calculer l'attach rate assurance par segment et de cibler les non-acheteurs.	Insurance attach rate par segment	H3 — Upsell
meal_purchase_flag	Booléen (0/1)	Achat de repas à bord. Pertinent surtout sur les vols régionaux et internationaux. Révèle l'appétit pour les ancillaires alimentaires par route_type.	Meal attach rate par route_type	H3 — Upsell
upgrade_flag	Booléen (0/1)	Le client a-t-il effectué un upgrade Economy → Business ? C'est le produit ancillaire à plus forte valeur unitaire. Permet de calculer le taux de conversion upgrade et d'identifier les profils réceptifs.	Upgrade conversion rate, Premium revenue	H3 — Upsell

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
promo_code_used	Booléen (0/1)	Réservation effectuée avec un code promo. Mesure la sensibilité prix par segment. Un segment très dépendant des promos indique une élasticité prix élevée — info cruciale pour la stratégie tarifaire et pour calibrer les offres upsell.	Prix-sensibilité par segment, Promo ROI	H3 + H1 (pricing)

4.2 Table Flights — Colonne ajoutée

Colonne	Type	Utilité analytique	KPI alimenté	Hypothèse H1/H2/H3
passengers_boarded	Mesure (INTEGER)	Nombre réel de passagers embarqués sur le vol. Différent de seat_capacity (capacité) et du nombre de réservations confirmées (à cause des no-shows). Permet de calculer le load factor réel (vs théorique basé sur bookings).	Load factor réel = $\frac{\text{passengers_boarded}}{\text{seat_capacity}}$	H1 — Réseau

Colonnes exclues des tables sources : customer_lifetime_value et churn_risk_score ne sont PAS ajoutés dans Customers. Ce sont des outputs analytiques calculés à partir des données brutes. Ils vivent dans un mart analytique (couche dbt) mis à jour régulièrement, pas dans une table source qui ne doit contenir que des faits observés.

5. Synthèse et couverture analytique

5.1 Couverture des hypothèses par dataset

Dataset / Colonne	H1 Réseau	H2 Rétention	H3 Upsell	Priorité
FlightCosts	Indispensable	—	—	P1
LoyaltyActivity	—	Indispensable	Partiel	P1
AncillaryDetail	—	Partiel	Indispensable	P1
CustomerReviews	Signal réputation	Cause churn	Friction ancillaire	P1
ComplaintLogs	Qualité route	Churn post-plainte	Produit défaillant	P1
SupportTickets	—	Friction temps réel	Ancillaire défaillant	P2
Bookings — flags (4 col)	Pricing signal	—	Attach rate direct	P2
Flights — passengers_boarded	Load factor réel	—	—	P2

5.2 Questions décisionnelles débloquées

Question business	Datasets requis	Réponse possible ?
Cette route génère-t-elle un profit réel ?	Bookings + FlightCosts	Oui

Question business	Datasets requis	Réponse possible ?
Quels clients à haute valeur risquent de churner ?	Customers + LoyaltyActivity + SupportTickets	Oui
Quel ancillaire pousser à quel segment ?	Bookings (flags) + AncillaryDetail + Customers	Oui
Pourquoi la satisfaction baisse sur la route X ?	CustomerReviews + ComplaintLogs + Flights (OTP)	Oui
Route non rentable : problème structurel ou opérationnel ?	FlightCosts + ComplaintLogs (type Delay) + Reviews	Oui
Où investir en priorité : H1, H2 ou H3 ?	Tous les datasets ci-dessus combinés	Oui — objectif final

PARTIE II : Justification d’architecture et décisions techniques

1. Contexte et contraintes du projet

Air Côte d'Ivoire (Air CI) est une compagnie aérienne régionale opérant 12 routes (domestic, régional, international) avec un dataset de 480 vols, 11 475 réservations et 300 clients sur la période janvier 2025.

Les 3 questions business à adresser par l'architecture :

- Route optimization & growth — Quelles routes développer, réduire ou reprendre ?
- Customer retention — Quels clients haute valeur sont à risque de churner ?
- Upsell / Cross-sell — Comment augmenter le revenu ancillaire par passager ?

2. Choix de stack technologique

La stack retenue : Python · DuckDB · dbt-core · Power BI · Cohere API.

Technologie	Rôle	Alternative écartée	Raison du choix
DuckDB	Stockage analytique OLAP	Snowflake / BigQuery	Zéro infrastructure, fichier unique .duckdb, performances OLAP excellentes sur ce volume, natif Parquet/Arrow, compatibilité dbt
dbt-core	Transformation & documentation	SQL pur / Spark	Standard industrie, lineage automatique, tests intégrés, snapshots SCD2 natifs, compatible DuckDB via adapter
Python	Orchestration & embeddings	Airflow / Dagster	Scope challenge ne justifie pas un orchestrateur. Scripts Python suffisants.
Power BI	Dashboard BI	Metabase / Looker	Connecteur DuckDB ODBC disponible, standard entreprise, capacités DAX pour KPIs complexes
Cohere API	Embeddings + LLM	OpenAI / Claude API	Free tier embed-multilingual-v3.0 couvre FR+EN. Tool use natif pour l'agent.

3. Choix d'architecture : Hybrid Model

Trois architectures ont été évaluées :

Architecture	Historisation	Flexibilité	Verdict
Star Schema simple	Pas de SCD2	Limitée	Insuffisant pour un profil Senior — cache les changements de données
Data Vault pur	Native SCD2	Très haute	Surconçu pour ce volume — complexité non justifiée
Hybrid RETENU	SCD2 via snapshots	Haute	Équilibre optimal : traçabilité + performance + lisibilité

Le Hybrid Model retenu combine :

- Raw Vault (Hubs + Links + Satellites) — pour la traçabilité source et l'historisation
- Business Vault (PIT + Bridge) — pour l'optimisation des jointures et la cohérence temporelle
- Star Schema Mart — pour la consommation BI optimisée
- Semantic Layer (vues) — pour la définition centralisée des KPIs
- OBT AI-Ready — pour les use cases ML et LLM

4. Justification du Raw Vault

4.1 Pourquoi des Hubs ?

Les hubs isolent les clés métier de leurs attributs. Quand une source change de nom de colonne ou de format, le hub reste stable — seul le satellite est impacté. Cette séparation garantit la pérennité du modèle face aux évolutions de source.

5 hubs ont été définis : `hub_flight`, `hub_customer`, `hub_route`, `hub_airport`, `hub_booking`. Chaque hub utilise SHA256 sur la clé métier pour produire un `hash_key` universel et indépendant de la source.

4.2 Pourquoi des Links ?

Les links modélisent les relations métier sans dupliquer les données. `lnk_booking` lie 3 hubs (`flight` + `customer` + `booking`) — ce qui en SQL naïf aurait nécessité des colonnes redondantes dans les deux sens. En Data Vault, la relation est centrale et autonome.

4.3 Pourquoi des Satellites avec `hash_diff` ?

Le `hash_diff` est le mécanisme central de détection des changements. À chaque run dbt, le hash des attributs est recalculé et comparé avec la valeur stockée. Si différence → nouvelle version (SCD2). Sinon → aucun insert inutile.

Exemple concret : si le `loyalty_tier` d'un client change de Silver à Gold, `sat_customer_profile` crée une nouvelle ligne avec `load_date` = aujourd'hui et ferme l'ancienne (`load_end_date`, `is_current=0`). L'historique complet est conservé.

5. Choix SCD2 : dbt Snapshots vs Incremental

Trois approches ont été évaluées pour implémenter le SCD2 :

Approche	SCD2 correct	Run unique	Verdict
Incremental merge sur <code>sat_hk</code>	Partiel (pas de close)		Ancien enregistrement jamais fermé → 2 lignes <code>is_current=1</code>
Incremental + <code>current_target</code>		(2 runs)	Nécessite 2 runs car <code>{{ this }}</code> inexistant au 1er run
dbt Snapshots RETENU			Natif dbt, gère <code>load_date/load_end_date/is_current</code> automatiquement

Les satellites lisent depuis les snapshots correspondants (`snap_*`) et mapent `dbt_valid_from` → `load_date`, `dbt_valid_to` → `load_end_date`.

6. Justification du Business Vault

6.1 Point-In-Time (PIT)

Problème sans PIT : reconstituer l'état d'un client à une date historique nécessite une jointure temporelle sur CHAQUE satellite avec des conditions `load_date` / `load_end_date`. Sur 3 satellites × 300 clients × 365 jours = calcul explosif.

Solution PIT : une ligne par (`hub_hk` × `snapshot_date`) qui stocke les `hash_keys` actifs de chaque satellite à cette date. Le mart fait 1 jointure sur le PIT et récupère les bons `hash_keys` — puis joint les satellites directement sur leurs `hash_keys`. $O(1)$ par satellite au lieu de $O(n)$ scans temporels.

3 PIT créés : `pit_flight` (3 satellites), `pit_customer` (2 satellites), `pit_route` (1 satellite, anticipation).

6.2 Bridge Tables

Problème sans Bridge : `fact_flights` doit traverser `hub_flight` → `lnk_flight_route` → `hub_route` → `lnk_booking` → `hub_booking` → `hub_customer` pour chaque vol. 5 jointures répétées à chaque requête BI.

Solution Bridge : `bridge_flight_booking` pré-calcule ce chemin une fois. Le mart joint le bridge et accède à tous les `hash_keys` nécessaires en 1 table. `bridge_customer_flight` fait la même chose avec un grain client — les deux bridges sont complémentaires et non redondants.

7. Justification du Star Schema Mart

Le mart lit depuis le Business Vault via PIT et Bridge — jamais directement depuis le STG. Cette règle garantit que :

- Toute donnée dans le mart est historisée (SCD2) et traçable
- Les marts restent cohérents temporellement (via PIT)
- L'ajout d'une nouvelle source impacte uniquement le vault, pas les marts

Les faits calculés dans les marts (`load_factor`, `yield`, `margin`) sont dérivés des mesures atomiques des satellites — ils ne stockent pas de données brutes dupliquées.

Exception documentée : `dim_aircraft` lit depuis `stg_flights` (valeurs `DISTINCT` sur `aircraft_type`) car `hub_aircraft` n'a pas été créé dans ce modèle. En production, on créerait `hub_aircraft` + `sat_aircraft_specs`.

8. Justification de la Semantic Layer

La semantic layer résout le problème de la définition inconsistante des KPIs. Sans elle, chaque analyste calcule le `load_factor` différemment — certains incluent les "Changed", d'autres non.

Les 4 vues sémantiques définissent une source de vérité unique pour 14 KPIs organisés en 3 blocs métier :

- Opérationnel (`sem_kpis_operations`) : `load_factor`, `OTP`, `delay_bucket`, `cancellation`
- Financier (`sem_kpis_revenue`) : `yield`, `RASK`, `CASK`, `margin`, `ancillary_share`
- Client (`sem_kpis_customer`) : `LTV`, `repeat_rate`, `no_show_rate`, `redemption_rate`
- Réseau (`sem_kpis_route`) : agrégation par route avec part du réseau

Les vues sont en `materialized=view` (non stockées) — elles se recalculent à chaque requête depuis les marts déjà optimisés. C'est le seul endroit légitime pour des vues dans l'architecture.

9. Justification des OBT AI-Ready

Les OBT répondent à un besoin spécifique : les modèles ML et les LLM performant mieux avec des tables larges et dénormalisées. Joindre 8 tables à chaque inférence est inacceptable en latence.

OBT	Colonnes	Usage principal
<code>obt_flight_full</code>	~45 colonnes	ML prédiction retard/yield, LLM contexte vol, dashboard exécutif
<code>obt_customer_360</code>	~40 colonnes	Segmentation ML, scoring rétention, LLM profiling client
<code>obt_route_summary</code>	~35 colonnes	Optimisation réseau, ML pricing, LLM network analysis, dashboard

Les labels ML (`recency_label`, `value_label`, `route_label_ontology`) sont pré-calculés dans les OBT pour éviter leur recalcul à chaque requête. Ces labels sont les prémices de la couche ontologie.

10. Justification de la couche Embeddings (Partie 4)

Les données non-structurées (plaintes, reviews, tickets) représentent un signal qualitatif que les KPIs structurés ne capturent pas. La recherche par mots-clés (`LIKE '%retard%'`) est insuffisante car un passager peut exprimer sa frustration sans utiliser le mot exact.

La stratégie retenue : Cohere `embed-multilingual-v3.0` (1024 dimensions, support FR+EN natif) génère des vecteurs sémantiques stockés dans DuckDB. La recherche utilise `array_cosine_similarity()` — natif DuckDB, sans vector store externe.

Les 3 sources embeddées (150 plaintes + 80 reviews + 120 tickets = 350 documents) sont suffisantes pour démontrer la valeur de l'approche sur ce dataset de challenge.

11. Trade-offs et limites documentées

Décision	Ce qu'on sacrifie	Justification
Star Schema dans les marts	Flexibilité data vault complète	Performance BI — les marts sont la couche de consommation, pas d'exploration
DuckDB fichier unique	Scalabilité multi-utilisateurs	Scope challenge — en production on migre vers Snowflake/BigQuery
Cohere vs Claude API	Tool use Claude natif pour la démo Claude.ai	Free tier Cohere couvre les 350 documents sans coût
208 coûts générés synthétiquement	Précision absolue des marges	Couverture 100% préférable à 62.5% pour des analyses cohérentes. Flag is_generated=1 maintenu.
PIT sur snapshot_date courant uniquement	Analyse rétrospective complète	Dataset limité à janvier 2025 — 1 seul snapshot_date en pratique
hub_aircraft absent	Modèle vault 100% complet	dim_aircraft lisible depuis STG sans historique — types d'appareils stables

12. Ordre d'exécution dbt

Étape	Commande	Résultat
1	dbt seed	Crée les tables raw_* depuis les fichiers CSV/Excel
2	dbt run --select staging	Crée les tables stg_* (nettoyage technique)
3	dbt snapshot	Crée les tables snap_* avec SCD2
4	dbt run --select vault	Crée hubs, links, satellites (lisent snap_*)
5	dbt run --select vault.business_vault	Crée pit_* et bridge_*
6	dbt run --select marts	Crée fact_* et dim_* (lisent Business Vault)
7	dbt run --select semantic	Crée les vues sem_kpis_*
8	dbt run --select ai_ready	Crée les OBT obt_*
9	python embed_generator.py	Génère les embeddings Cohere → complaint_embeddings
10	dbt test	Valide toutes les couches (unique, not_null, accepted_values)

PARTIE III : Architecture data, Power BI et Agentic AI

Cette section accompagne le projet dbt, DuckDB, Power BI et interface IA developpe pour le challenge Analytics Engineer.

1. Objectif du document

Ce document explique les choix d'architecture retenus pour transformer les donnees Air Cote d'Ivoire en produits analytiques exploitables, puis les exposer a une interface d'IA agentique.

Il ne s'agit pas d'un guide d'installation, mais d'un document de conception. Il justifie les couches data, le modele analytique, le role des tables AI-ready, le modele Power BI et le fonctionnement de l'agent IA.

2. Contexte metier

Air Cote d'Ivoire doit arbitrer ses investissements de croissance entre plusieurs leviers : reseau, rentabilite, qualite operationnelle, retention client et revenus additionnels.

Le dashboard attendu doit permettre aux decideurs de repondre a une question principale : ou allouer le budget de croissance pour maximiser la marge, proteger les clients a forte valeur et developper les opportunités d'upsell et cross-sell ?

Pour repondre a cette question, le projet combine une architecture data fiable, un modele BI lisible et une couche IA capable d'interroger les donnees en langage naturel.

3. Vue d'ensemble de l'architecture

L'architecture suit une logique en couches : sources CSV, staging, Data Vault, Business Vault, marts, semantic, ontology, AI-ready, puis consommation par Power BI et par l'Agentic AI.

Chaque couche a un role distinct. Les couches basses assurent la qualite, la tracabilite et l'historisation. Les couches hautes portent les KPIs, les labels decisionnels et les tables optimisees pour l'analyse executive.

Flux logique : sources CSV -> staging -> Data Vault -> Business Vault -> marts -> semantic -> ontology -> AI-ready -> Power BI / Agentic AI.

4. Justification des couches data

La couche staging standardise les sources : noms de colonnes, types, montants, statuts, indicateurs et premieres regles de qualite. Elle evite d'exposer directement les fichiers bruts aux analyses.

La couche Data Vault separe les entites metier, les relations et les attributs historisables. Cette approche facilite la tracabilite, l'evolutivite et l'ajout de nouvelles sources sans casser les couches de restitution.

La couche Business Vault reconstruit des vues metier plus pratiques a partir du Vault : tables PIT, bridges et aggregations relationnelles. Elle sert de transition entre le modele historise et les modeles analytiques.

Les marts fournissent un schema analytique classique : dimensions et faits. C'est la couche la plus adaptee a la BI parce qu'elle est comprehensible, stable et performante pour les aggregations.

La couche semantic centralise les KPIs metier dans dbt. Elle documente la definition des indicateurs comme le load factor, l'OTP, la marge, le RASK, le CASK, le LTV et les taux d'ancillary.

La couche ontology transforme les KPIs en labels decisionnels : route rentable, route deficiente, client a risque, client high-value, opportunité upsell. Elle rapproche les donnees de l'action metier.

La couche AI-ready produit des tables denormalisees pretes pour l'IA et l'analyse executive : obt_route_summary, obt_customer_360 et obt_flight_full.

Synthese des couches

Couche	Rôle dans le projet
Staging	Standardisation des sources, typage, nettoyage technique et contrôles de qualité.
Data Vault	Traçabilité, historisation et séparation des hubs, links et satellites.
Business Vault	PIT, bridges et agrégations métier pour faciliter les jointures et la cohérence temporelle.
Marts	Schéma en étoile lisible et performant pour la BI.
Semantic	Définition centralisée des KPIs et conventions de nommage.

Ontology	Labels décisionnels et règles d'inférence orientées action.
AI-ready	OBT dénormalisées pour l'IA, le ML, les analyses rapides et la restitution executive.

5. Pourquoi utiliser des OBT

Les OBT, ou One Big Tables, ne remplacent pas le modèle analytique. Elles servent de tables de synthèse pour des usages précis : IA, ML, analyse rapide et restitution executive.

obt_route_summary contient une ligne par route et regroupe les KPIs réseau, opérationnels, financiers et de satisfaction. Elle est utile pour les questions de budget et de rentabilité.

obt_customer_360 contient une ligne par client et regroupe valeur, comportement, fidélité, recensement, fréquence et signaux de satisfaction. Elle est utile pour la rétention et l'upsell.

obt_flight_full contient une ligne par vol. Elle est utile pour l'analyse détaillée ou les cas ML, mais elle n'est pas nécessaire dans le modèle Power BI principal car elle duplique les faits et dimensions.

6. Choix du modèle Power BI

Le modèle Power BI recommandé est un schéma en étoile centré sur les faits : vols, réservations, coûts de vol et activité fidélité.

Les dimensions principales sont : dates, routes, clients, aircraft, aéroport origine et aéroport destination.

Les tables sémantiques ne sont pas importées dans Power BI afin d'éviter la redondance avec les faits, les mesures DAX et les OBT de synthèse.

Les tables staging et vault ne sont pas importées car elles sont techniques. Elles servent à construire la qualité et la traçabilité, pas à la restitution executive.

Les tables décisionnelles importées dans Power BI sont obt_route_summary, obt_customer_360, ont_route_labels, ont_customer_labels et ont_upsell_opportunities.

Tables Power BI recommandées

Catégorie	Tables à importer / exclure
Dimensions	dim_date, dim_route, dim_customer, dim_aircraft, dim_airport_origin, dim_airport_destination.
Facts	fact_flights, fact_bookings, fact_flight_costs, fact_loyalty_activity.
Decision	obt_route_summary, obt_customer_360, ont_route_labels, ont_customer_labels, ont_upsell_opportunities.
Exclues	staging, vault, semantic views et tables techniques non destinées à la restitution executive.

7. Role de la couche Agentic AI

L'Agentic AI ajoute une interface conversationnelle au-dessus des données analytiques. L'utilisateur peut poser une question métier en langage naturel, sans connaître les tables ni le SQL.

L'agent ne répond pas uniquement avec le modèle de langage. Il sélectionne des outils analytiques, interroge DuckDB, récupère des résultats structurés, puis demande au LLM de produire une réponse lisible et actionnable.

Cette approche limite les hallucinations car les chiffres et recommandations sont ancrés dans les tables dbt produites par le pipeline.

8. Fonctionnement de l'Agentic AI

Le fonctionnement suit cinq étapes.

Étape 1 : l'utilisateur pose une question dans l'interface React ou en ligne de commande via llm_agent.py.

Étape 2 : llm_agent.py analyse l'intention de la question. Par exemple, une question sur le budget route déclenche l'outil get_budget_recommendation ; une question sur la satisfaction d'une route déclenche analyze_route_sentiment.

Étape 3 : l'agent appelle les fonctions analytiques exposées dans mcp_server.py. Dans l'application actuelle, mcp_server.py est importé comme bibliothèque Python ; il n'est pas nécessaire de lancer un serveur MCP séparé pour l'interface.

Etape 4 : les outils interrogent DuckDB, notamment `main_ai_ready.obt_route_summary`, `main_ai_ready.obt_customer_360` et `main.complaint_embeddings`.

Etape 5 : les resultats sont transmis au modele Cohere, qui formule une reponse executive structuree avec contexte, analyse, recommandations et impact attendu.

Composants Agentic AI

Composant	Rôle
<code>airci_interface.py</code>	Backend Flask ou FastAPI qui expose l'endpoint de question-réponse.
<code>ui/</code>	Interface React pour poser les questions métier.
<code>llm_agent.py</code>	Analyse l'intention, choisit les outils et orchestre la réponse.
<code>mcp_server.py</code>	Couche d'outils analytiques interrogeant DuckDB et les embeddings.
<code>dev.duckdb</code>	Base analytique locale contenant les tables dbt et AI-ready.
Cohere	LLM et modèle d'embeddings multilingue FR/EN.
<code>main.complaint_embeddings</code>	Table vectorisée utilisée pour la recherche sémantique dans les plaintes.

9. Outils analytiques disponibles

`get_route_performance` : recupere les indicateurs de performance d'une route.

`compare_routes` : compare deux routes sur la rentabilite, la ponctualite, la demande et la satisfaction.

`get_budget_recommendation` : identifie les routes qui meritent plus de budget ou une optimisation.

`get_at_risk_customers` : liste les clients a risque de churn ou d'inactivite.

`analyze_route_sentiment` : explique les facteurs de faible satisfaction sur une route.

`search_complaints` : effectue une recherche semantique dans les plaintes vectorisees.

10. Flux technique de bout en bout

Interface React -> POST /api/ask -> `airci_interface.py` -> `llm_agent.py` -> outils `mcp_server.py` -> DuckDB / Cohere -> reponse formatee dans l'interface.

Le fichier `.env` contient la cle `COHERE_API_KEY`. Le modele Cohere peut etre configure avec `COHERE_CHAT_MODEL`.

DuckDB sert de moteur analytique local. dbt construit les tables et garantit leur documentation et leurs tests. L'agent consomme ces tables comme source de verite.

11. Recherche semantique et embeddings

Les plaintes clients sont vectorisees afin de permettre une recherche semantique. Cela permet de retrouver les plaintes proches d'une question meme si les mots exacts ne sont pas identiques.

La table `main.complaint_embeddings` contient les textes de plaintes et leurs vecteurs. Elle est utilisee pour enrichir les analyses de satisfaction et identifier les causes de mecontentement.

Cette couche est particulierement utile pour expliquer pourquoi une route a un faible sentiment client : bagages, service, retard, remboursement, communication ou incidents.

12. Qualite, gouvernance et tracabilite

La qualite est geree par dbt via les tests `not_null`, `unique`, `accepted_values`, `relationships` et les tests `dbt_utils`.

La documentation dbt genere les artefacts `target/catalog.json`, `target/manifest.json` et `target/index.html`.

Le dictionnaire de donnees `DATA_DICTIONARY.md` et sa version Excel `DATA_DICTIONARY.xlsx` complètent la documentation technique en expliquant les tables et les colonnes metier.

La separation des couches rend le projet auditable : une erreur peut etre localisee entre source, staging, vault, mart, semantic, ontology ou AI-ready.

13. Limites et precautions

Les reponses de l'IA dependent de la qualite des tables dbt et de la disponibilite de l'API Cohere.

Les OBT sont pratiques pour l'IA, mais elles ne doivent pas remplacer l'ensemble du modele en etoile dans Power BI.

Les tables semantic ne sont pas importees dans Power BI pour eviter la duplication, mais elles restent importantes dans dbt pour documenter les definitions de KPI.

Les recommandations de l'agent doivent etre interpretees comme une aide a la decision. Les arbitrages finaux doivent rester sous controle metier.

14. Evolution possible

Connecter l'agent a un vrai serveur MCP externe si un client compatible MCP doit consommer les outils.

Ajouter une couche de monitoring des questions posees et des outils appeles.

Ajouter des controles de securite par role afin de limiter l'accès a certaines donnees clients.

Automatiser la generation des embeddings apres chaque nouveau chargement de plaintes.

Publier le rapport Power BI avec une page executive finale et un walkthrough video.

15. Conclusion

L'architecture proposee combine robustesse data, lisibilite BI et capacite IA.

dbt assure la transformation, la qualite et la documentation. DuckDB fournit un moteur analytique local simple et performant. Power BI restitue la vision executive. L'Agentic AI rend les analyses accessibles en langage naturel.

Cette separation claire permet de couvrir les besoins du challenge : network and profitability, customer and retention, upsell and cross-sell, et couche finale de recommandations.